

Linguagem e Técnicas de Programação

Professor Msc. Aparecido Vilela Junior
aparecido.vilela@unicesumar.edu.br

O que são arquivos?

Os arquivos são estruturas de dados manipuladas fora do ambiente do programa.

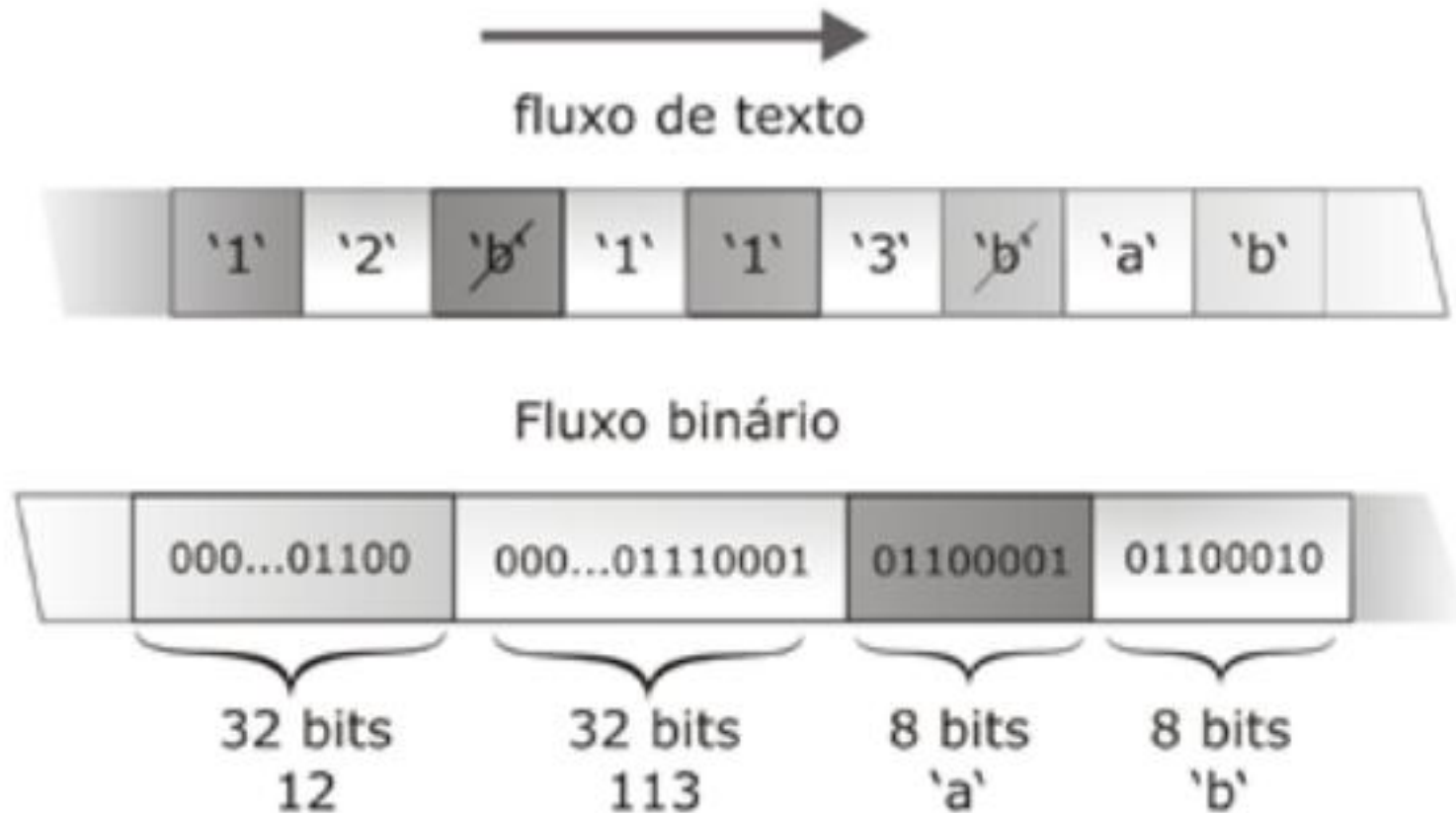
Considera-se como ambiente do programa a memória principal, onde nem sempre é conveniente manter certas estruturas de dados.

De modo geral, os arquivos são armazenados na memória secundária, como, por exemplo: disco rígido (HD - *hard disk*), *CD* e *pendrive*.

Como os arquivos são organizados?

- ▶ A linguagem C utiliza o conceito de fluxo de dados (***stream***) para manipular os vários tipos de dispositivos de armazenamento e seus diferentes formatos.
- ▶ Os dados podem ser manipulados em dois diferentes tipos de fluxos: fluxos de texto e fluxos binários.
- ▶ Um fluxo de texto (*text stream*) é composto por uma sequência de caracteres, que pode ou não ser dividida em linhas, terminadas por um caractere de final de linha.
- ▶ Um fluxo binário (*binary stream*) é composto por uma sequência de *bytes*, que são lidos, sem tradução, diretamente do dispositivo externo.

Fluxo de Dados



Fluxo de Dados

- ▶ No fluxo de texto, os dados são armazenados como caracteres sem conversão para a representação binária.
- ▶ Cada um dos caracteres ocupa um *byte*. *O número 12 ocupa dois bytes e o número 113 ocupa 3 bytes.*
- ▶ Um caractere em branco foi inserido entre cada um dos números para separá-los, de modo que a função de entrada e saída possa descobrir que são dois números inteiros (12 e 113) e não o número 12113.
- ▶ No fluxo binário, cada número inteiro ocupa 32 *bits* (4 *bytes*) e é armazenado na forma binária. Observem que, em arquivos binários, não há necessidade de separar os números já que eles sempre ocupam 32 *bits*.
- ▶ Os arquivos binários são utilizados quando queremos armazenar registros completos.
- ▶ Com estes arquivos, poderemos acessar qualquer registro de forma mais rápida.
- ▶ No caso dos arquivos texto, quando queremos encontrar alguma informação, temos que fazer uma varredura seqüencial no arquivo, tornando a busca pouco eficiente.

Ponteiros

Um ponteiro é um tipo de variável que armazena um endereço de memória.

Nós já vimos variáveis que armazenam números inteiros, números reais e caracteres.

Ao trabalhar com arquivos, precisamos saber em qual endereço de memória o arquivo está armazenado.

O endereço de memória onde o arquivo está armazenado será colocado em uma variável que armazena endereço de memória (ponteiro).

Ponteiros

- ▶ Para declarar uma variável que é capaz de armazenar um endereço de memória, usamos a seguinte sintaxe:

Sintaxe

```
tipo *nome_do_ponteiro;
```

- ▶ Onde:
 - **Tipo:** Tipo de variável que o ponteiro armazena endereço. Podemos dizer que nosso ponteiro armazena o endereço de uma variável inteira, real, caractere, etc. Para este capítulo, estaremos utilizando um ponteiro que vai armazenar o endereço de um arquivo.
 - *****: o asterisco na frente do nome de uma variável simboliza que a variável que está sendo declarada é um ponteiro.
 - **nome_do_ponteiro:** daremos um nome à nossa variável que armazena endereços.

Declaração de ponteiros para arquivos

```
FILE *fagenda;
```

- ▶ Na linha 1, temos a declaração do ponteiro *fagenda*, que irá armazenar o endereço de um arquivo (FILE).
- ▶ Devemos colocar o FILE em maiúsculo.
- ▶ Quando queremos inicializar uma variável real ou inteira, atribuímos zero às mesmas.
- ▶ Quando precisarmos inicializar um ponteiro, devemos atribuir: NULL. Quando um ponteiro armazena NULL, quer dizer que ele não está armazenando um endereço no momento.
- ▶ Vamos começar a conhecer os comandos de manipulação de arquivos binários e entenderemos melhor onde o conceito de ponteiro será aplicado.

Comandos para manipular arquivos

Binários

Como já mencionado, os arquivos binários são utilizados quando queremos armazenar registros.

É como se tivéssemos um vetor de registro, só que os dados não são perdidos ao terminar a execução do programa.

Representação gráfica de um arquivo binário



Comandos para manipular arquivos Binários

O arquivo binário é formado por um conjunto de registros, armazenados um após o outro. As operações realizadas em um arquivo binário dependerão do local onde se encontrar o **leitor**.

Comandos para manipular arquivos

Binários

- ▶ O leitor do arquivo passará sobre os registros, fazendo a leitura dos mesmos.
- ▶ Nós podemos colocar o leitor sobre qualquer registro e executar uma operação sobre o mesmo (leitura e/ou gravação).
- ▶ O arquivo tem uma marcação indicando onde ele termina (*end of file*).
- ▶ As operações para a manipulação de arquivo estão na biblioteca *stdio.h*. Com isso, ao trabalhar com arquivos, devemos incluir esta biblioteca nos nossos programas.

Declaração de um ponteiro para arquivo

Na linguagem C, as funções que manipulam arquivos trabalham com o conceito de ponteiros para arquivo.

Com isso, teremos uma variável que armazenará o endereço de memória onde está armazenado nosso arquivo.

Devemos declarar um ponteiro para cada arquivo que formos manipular no nosso programa.

Comando para abrir um arquivo

- ▶ A maior parte das operações sobre um arquivo (leitura, gravação, etc) só pode ser executada com o arquivo aberto. O comando de abertura do arquivo (*fopen*):

Sintaxe

```
ponteiro_arquivo = fopen("nome_do_arquivo",  
"modo_de_abertura");
```

- ▶ Onde:
 - **ponteiro_arquivo:** Ao abrir um arquivo, a função *fopen* retorna o endereço de memória do arquivo. Por conta disso, devemos atribuir o endereço de memória do arquivo, para um ponteiro. Após o arquivo ser aberto, usaremos este endereço de memória para acessá-lo.
 - **nome_do_arquivo:** Determina qual arquivo deverá ser aberto. Neste espaço é colocado o nome do arquivo, da forma que foi salvo no diretório. O comando *fopen* procura o arquivo que desejamos abrir, no mesmo diretório onde está armazenado o programa executável.
 - **modo_de_abertura:** Informa que tipo de uso você vai fazer do arquivo. O modo de abertura indica o que faremos no arquivo: leitura, gravações e alterações. Além disso, o modo de abertura também vai indicar se queremos que um novo arquivo seja criado.

Modos de Abertura

Modo	Significado
"r+b"	Abre um arquivo binário para leitura e escrita. O arquivo deve existir antes de ser aberto. Dessa forma, este modo de abertura só pode ser usado se o arquivo que estamos querendo abrir já existe no nosso computador.
"w+b"	Cria um arquivo binário para leitura e escrita. Se o arquivo não existir, ele será criado. Se já existir, o conteúdo anterior será destruído. Este modo de abertura tem a capacidade de criar novos arquivos. Mas, se solicitarmos que seja aberto um arquivo que já existe, o conteúdo do arquivo será apagado.
"a+b"	Acrescenta dados ou cria um arquivo binário para leitura e escrita. Caso o arquivo já exista, novos dados podem ser adicionados, apenas, no final do arquivo. Se o arquivo não existir, um novo arquivo será criado.

Abertura do Arquivo

```
fagenda = fopen("arquivo_agenda_binario.txt", "w+b");  
  
if (fagenda != NULL) {  
    printf("\n Arquivo criado com sucesso!");  
    fclose(fagenda);  
}  
else  
    printf("\n Erro na criação do arquivo!");
```

- ▶ Para saber se um arquivo foi aberto corretamente, podemos fazer um teste como mostra as linhas 2 e 6.
- ▶ Após executar um *fopen*, verificamos se o ponteiro está com *NULL*. Caso afirmativo, é porque a abertura não foi executada corretamente.
- ▶ Uma vez aberto, o arquivo fica disponível para leituras e gravações através da utilização de funções adequadas.

Comando para fechar um arquivo

- ▶ Após terminar de usar um arquivo, devemos fechá-lo. Para isso, usamos a função *fclose*, que tem a seguinte sintaxe.

Sintaxe
<code>fclose(ponteiro_arquivo);</code>

- ▶ Onde:
 - **ponteiro_arquivo:** É o ponteiro que tem armazenado o endereço de memória do arquivo que queremos fechar.

Só podemos fechar arquivos que estão abertos.

Fechamento de arquivo

```
fclose(fagenda);
```

- ▶ Na linha 1, fechamos o arquivo que está armazenado no endereço de memória *fagenda* (que é o ponteiro que armazena o endereço do arquivo).
- ▶ Nós só utilizamos o nome do arquivo (o nome que está salvo no diretório), no momento da abertura. Depois de aberto, só utilizamos seu endereço de memória, certo?

Comando para ler um registro armazenado no arquivo

- ▶ A leitura é feita a partir do ponto onde o leitor se encontra.
- ▶ Ao abrir o arquivo, o leitor é posicionado no início do primeiro registro.
- ▶ À medida que vamos executando leituras, o leitor vai se deslocando.
- ▶ Dessa forma, precisamos ter noção da posição onde o leitor se encontra.
- ▶ Nos arquivos binários, SEMPRE fazemos a leitura de um registro completo, independente de quantos campos ele tenha.
- ▶ Um arquivo deve armazenar registros do mesmo tipo. Se os registros são do mesmo tipo, cada um deles ocupará o mesmo espaço de memória.
- ▶ Dessa forma, o leitor sempre se deslocará em intervalos regulares.

Leitura do arquivo

sintaxe

```
fread (&registro, numero_de_bytes, quantidade,  
ponteiro_arquivo);
```

► Onde:

- **®istro:** É o registro que armazenará os dados do registro lido do arquivo. É assim, nós pegamos um registro que está no arquivo, e armazenamos no registro passado como parâmetro.
- **numero_de_bytes:** O que o comando de leitura faz é informar que o leitor precisa se deslocar, fazendo a leitura de uma certa quantidade de bytes. A quantidade de bytes que o leitor deve se deslocar é exatamente quantos bytes de memória o registro ocupa. Por exemplo: se o nosso arquivo armazena registros de contatos de pessoas, composto pelos campos: código (int), idade (int), telefone (int), sexo (char) e nome (char[50]) é importante usar o comando `sizeof` que calcula e informa a totalidade de bytes para armazenar um tipo de dado. Com isso será identificado o número de bytes que o leitor deverá ler, para recuperar o registro de um contato, armazenado no arquivo. Para utilizar a função `sizeof`, precisamos apenas, passar como parâmetro para a função, que tipo de dado queremos saber quantos *bytes* ele ocupa. Por exemplo: *`sizeof(int)` ou `sizeof(contato)`*. No caso de um registro, a função vai fazer as contas de quantos bytes cada um dos campos do registro ocupa, e retorna a soma.

Leitura do arquivo

sintaxe

```
fread (&registro, numero_de_bytes, quantidade,  
ponteiro_arquivo);
```

- ▶ **Quantidade:** Neste terceiro parâmetro, usaremos **SEMPRE** 1. Este parâmetro significa quantas vezes a leitura será executada. Nós sempre queremos que seja feita a leitura de um registro por vez. Por isso, sempre utilizaremos 1.
- ▶ **Ponteiro_arquivo:** Precisamos informar em qual arquivo a leitura será feita. Com isso, passamos o endereço de memória onde o arquivo que será lido está armazenado. Assim, utilizamos o ponteiro que tem o endereço do arquivo.

Exemplo

```
typedef struct {  
    int codigo, idade, telefone;  
    char sexo;  
    char nome[50];  
} contato;  
  
contato pessoa;  
  
FILE *fagenda;  
  
fread(&pessoa, sizeof(contato), 1, fagenda);
```

Arquivo Binário



Posição do leitor antes de executar o fread



Posição do leitor após de execução do fread

Comando para gravar um registro no arquivo

Para gravar um registro em um arquivo binário, nós utilizamos o comando *fwrite*.

Este comando é muito parecido com o fread.

A diferença é que o *fread* lê uma sequência de bytes no arquivo, e armazena em um registro (primeiro parâmetro do *fread*).

E o *fwrite*, pega um registro, e armazena suas informações no arquivo.

Comando para gravar um registro no arquivo

- ▶ Vamos ver a sintaxe do *fwrite*:

Sintaxe

```
fwrite(&registro, numero_de_bytes, quantidade,  
ponteiro_arquivo);
```

- ▶ Onde:
 - **®istro:** é o registro que será armazenado no arquivo.
 - **numero_de_bytes:** é a quantidade de *bytes que serão* gravadas no arquivo. Neste parâmetro, também usaremos o *sizeof*.
 - **quantidade:** neste terceiro parâmetro, também, usaremos SEMPRE 1. Este parâmetro significa quantas vezes a gravação será executada. Nós sempre queremos que seja feita a gravação de um registro por vez. Por isso, sempre utilizaremos 1.
 - **ponteiro_arquivo:** precisamos informar em qual arquivo a gravação será feita. Assim, utilizamos o ponteiro que tem o endereço do arquivo.

Exemplo: Leitura de um registro do arquivo

```
typedef struct {  
    int codigo, idade, telefone;  
    char sexo;  
    char nome[50];  
} contato;  
  
contato pessoa;  
  
FILE *fagenda;  
  
fwrite(&pessoa, sizeof(contato), 1, fagenda);
```

Comando para verificar se chegou ao final do arquivo

Quando fazemos uma varredura no arquivo, não sabemos quantos registros tem armazenados no mesmo.

No entanto, precisamos saber o momento de parar de executar a leitura dos registros.

O comando *feof (end of file)* informa se o leitor chegou ao final do arquivo ou não.

Comando para verificar se chegou ao final do arquivo

Sintaxe

```
int feof(ponteiro_arquivo);
```

► Onde:

- **int: é o retorno da função.** A função *feof* retorna um número inteiro, que indica se o arquivo terminou ou não. Quando a função retorna zero, significa que ainda não chegou no final do arquivo. Qualquer valor diferente de zero, indica que chegou ao final do arquivo.
- **ponteiro_arquivo:** é o ponteiro que tem o endereço do arquivo que queremos verificar se chegou ao fim.

Exemplo

Normalmente, o *feof* é usado como condição de um *while*. Este *while* será executado enquanto não chegar no final do arquivo.

```
while (feof(fagenda)==0)
```

Comando para remover um arquivo

Quando desejamos apagar um arquivo do diretório, podemos utilizar o comando *remove*.

Este comando só pode ser utilizado em arquivos fechados.

Sintaxe

```
remove("nome_do_arquivo");
```

Onde:

nome_do_arquivo: é o nome do arquivo que queremos apagar. Neste caso, é utilizado o arquivo no diretório do nosso computador.

Exemplo

```
remove("agenda.bin");
```

Comando para renomear um arquivo

Nós podemos renomear um arquivo no diretório do nosso computador. Para isso, utilizamos o comando *rename*, só pode ser utilizado em arquivos fechados.

Sintaxe

```
rename("nome_do_arquivo", "novo_nome_do_arquivo");
```

Onde:

nome_do_arquivo: é o nome do arquivo que queremos renomear. Neste caso; é utilizado o nome do arquivo salvo no diretório do nosso computador.

novo_nome_do_a

Exemplo

```
rename("agenda.bin", "agenda_temp.bin");
```

Neste exemplo, o arquivo *agenda.bin*, será renomeado para *agenda_temp.bin*.